Neural Networks

Comprehensive Cheatsheet

Machine Intelligence - Andrés Masegosa - Fall 2025

Contents

1 From Single Neurons to Networks **2**

1.1 The Journey: Logistic Regression to Neural Networks 2

1.2 Linearly Separable vs Non-Linearly Separable 3

2 The Perceptron **4**

2.1 Perceptron Architecture 4

2.2 Perceptron Inference (Making Predictions) 5

2.3 Perceptron Learning Algorithm 6

2.4 Perceptron in Python 9

3 Feed-Forward Neural Networks (FFNNs) **11**

3.1 Architecture and Terminology 11

3.2 Mathematical Formulation 13

4 Activation Functions **15**

4.1 Why Activation Functions? 15

4.2 Common Activation Functions 15

5 Training Neural Networks **18**

5.1 Loss Functions 18

5.2 Gradient Descent and Backpropagation 19

5.3 Optimization Algorithms 20

6 Implementation in Python **21**

6.1 Using TensorFlow/Keras 21

6.2 Complete FFNN Example: MNIST Digits - Part 1 22

6.3 Complete FFNN Example: MNIST Digits - Part 2 22

6.4 Detailed Layer Explanation 23

6.5 Data Preprocessing 24

6.6 Model Training Tips 24

7 Evaluation Metrics **26**

7.1 Beyond Accuracy 26

7.2 Computing Metrics in Python 27

8 Common Issues and Solutions **28**

8.1 Overfitting 28

8.2 Underfitting 30

9 What Have You Learned? **31**

1 From Single Neurons to Networks

1.1 The Journey: Logistic Regression to Neural Networks

Evolution of Machine Learning Models

The Progression:

- **Linear Regression**: Predicts continuous values
  - Model: $f(\vec{x}) = \vec{w} \cdot \vec{x} + b$
  - Can only model linear relationships
- **Logistic Regression**: Binary classification
  - Model: $f(\vec{x}) = \sigma(\vec{w} \cdot \vec{x} + b)$
  - Adds sigmoid to constrain output to $[0, 1]$
  - Still fundamentally linear decision boundary
- **Perceptron**: Simplest neural network
  - Single artificial neuron
  - Binary classification with simple learning rule
  - Limited to linearly separable problems
- **Feed-Forward Neural Networks (FFNNs)**: Multiple neurons
  - Multiple layers of neurons
  - Can learn non-linear decision boundaries
  - Universal function approximators (in theory)

Key Limitation of Single Neurons: They can only learn linearly separable patterns. Real-world problems are often non-linear!

1.2 Linearly Separable vs Non-Linearly Separable

Linear Separability

Definition: A dataset is **linearly separable** if you can draw a straight line (in 2D), plane (in 3D), or hyperplane (in higher dimensions) that perfectly separates the two classes.
Linearly Separable Examples:

- Two clusters of points with clear separation
- AND, OR, NOT logical operations

NOT Linearly Separable Examples:

- XOR problem (classic example)
- Circular clusters (points inside vs outside a circle)
- Interleaved patterns
- Most real-world problems!

Plain English: If you can separate your data with a straight line/plane, use a perceptron. If not, you need a neural network!

The XOR Problem

XOR Truth Table:

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Why a perceptron fails:

- Try to draw a straight line that separates (0,0) and (1,1) from (0,1) and (1,0)
- It's impossible! The pattern is non-linear
- This was proven by Minsky and Papert (1969)
- Caused the first "AI winter" (funding drought)

Solution: Use a neural network with at least one hidden layer!

2 The Perceptron

2.1 Perceptron Architecture

Perceptron: The Simplest Neural Network

Components:

- **Inputs**: $\vec{x} = [x_1, x_2, \ldots, x_n]$
- **Weights**: $\vec{w} = [w_1, w_2, \ldots, w_n]$ (learned parameters)
- **Bias**: b (learned parameter)
- **Activation**: Weighted sum + decision threshold
- **Output**: Binary decision (Yes/No, 1/0, +/-)

Architecture Diagram:

$$\begin{array}{c} x_1 \xrightarrow{w_1} \\ x_2 \xrightarrow{w_2} \rightarrow \text{[Neuron with bias } b\text{]} \rightarrow \text{decision} \\ x_n \xrightarrow{w_n} \end{array}$$

Biological Inspiration:

- Loosely inspired by neurons in the brain
- Inputs = dendrites receiving signals
- Weights = synaptic strengths
- Activation = neuron firing threshold
- Output = axon transmitting signal

Important Note: This is just inspiration! Artificial neural networks are NOT accurate models of the brain.

2.2 Perceptron Inference (Making Predictions)

Perceptron Decision Function

Step 1: Compute weighted sum (dot product):

$$z = \vec{w} \cdot \vec{x} + b = \sum_{i=1}^{n} w_i x_i + b$$

Step 2: Apply decision threshold:

$$\text{decision}(x) = \begin{cases} \text{Yes (or 1)} & \text{if } z > 0 \\ \text{No (or 0)} & \text{if } z \leq 0 \end{cases}$$

Plain English:

1. Multiply each input by its weight
2. Add all products together
3. Add the bias
4. If result is positive → predict Yes
5. If result is negative or zero → predict No

Decision Boundary: The hyperplane where $\vec{w} \cdot \vec{x} + b = 0$

- Points on one side → Yes
- Points on other side → No
- Points exactly on the boundary → No (by convention)

Perceptron Inference Example

Given:

- $\vec{x} = [2, 1, 0]$ (input features)
- $\vec{w} = [5, -2, 3]$ (weights)
- $b = 1$ (bias)

Step 1: Compute dot product:

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + w_3x_3 + b \\ &= 5(2) + (-2)(1) + 3(0) + 1 \\ &= 10 - 2 + 0 + 1 \\ &= 9 \end{aligned}$$

Step 2: Make decision:

- Since $z = 9 > 0 \rightarrow$ Predict **Yes**

If we changed the input to $\vec{x} = [0, 3, 0]$:

$$\begin{aligned} z &= 5(0) + (-2)(3) + 3(0) + 1 \\ &= 0 - 6 + 0 + 1 \\ &= -5 \end{aligned}$$

Since $z = -5 < 0 \rightarrow$ Predict **No**

2.3 Perceptron Learning Algorithm

Error-Driven Learning

Core Idea: Only update weights when we make a mistake!
Types of Mistakes:

1. **False Negative (FN)**:
   - Actual label: Yes
   - Predicted: No
   - Problem: Weights too small $\rightarrow$ increase them
2. **False Positive (FP)**:
   - Actual label: No
   - Predicted: Yes
   - Problem: Weights too large $\rightarrow$ decrease them

Why this works: Each correction moves the decision boundary closer to correctly classifying the example.

Perceptron Learning Algorithm

Algorithm:
Initialize:

$$\vec{w} = \vec{0} \quad \text{(all weights start at 0)}$$
$$b = 0 \quad \text{(bias starts at 0)}$$

Repeat until convergence:

1. **For each training example** $(x^{(i)}, y^{(i)})$:
2. **Make prediction**: $\hat{y} = \text{decision}(\vec{x}^{(i)}, \vec{w}, b)$
3. **Check if correct**:
   - If $\hat{y} == y^{(i)}$: Skip to next example (no update needed!)
4. **If False Negative** ($y^{(i)} = \text{Yes}$ but $\hat{y} = \text{No}$):

$$b = b + 1$$
$$\vec{w} = \vec{w} + \vec{x}^{(i)}$$

5. **If False Positive** ($y^{(i)} = \text{No}$ but $\hat{y} = \text{Yes}$):

$$b = b - 1$$
$$\vec{w} = \vec{w} - \vec{x}^{(i)}$$

Convergence: Algorithm stops when all training examples are correctly classified (if data is linearly separable) or after maximum iterations.

Perceptron Learning Example: Finding Mom

Dataset: Animal classification ("Is this my mom?")
Features: [isBig, hasTail, hasTrunk, hasColorBrown, numberOfLimbs]
Training Data:

isBig	hasTail	hasTrunk	Brown	Limbs	Label
1	1	1	0	4	No (Elephant)
0	1	0	0	0	No (Snake)
0	1	0	1	4	Yes (Monkey)

Initial state: $\vec{w} = [0, 0, 0, 0, 0]$, $b = 0$
Epoch 1:
Example 1 (Elephant):

- $z = 0 \cdot [1, 1, 1, 0, 4] + 0 = 0$
- Prediction: No (correct!)
- No update needed

Example 2 (Snake):

- $z = 0 \cdot [0, 1, 0, 0, 0] + 0 = 0$
- Prediction: No (correct!)
- No update needed

Example 3 (Monkey):

- $z = 0 \cdot [0, 1, 0, 1, 4] + 0 = 0$
- Prediction: No, but should be Yes
- False Negative! Update:

$$b = 0 + 1 = 1$$
$$\vec{w} = [0, 0, 0, 0, 0] + [0, 1, 0, 1, 4] = [0, 1, 0, 1, 4]$$

After Epoch 1: $\vec{w} = [0, 1, 0, 1, 4]$, $b = 1$
Now the perceptron has learned:

- Positive weight for hasTail (mom has tail)
- Positive weight for hasColorBrown (mom is brown)
- Positive weight for numberOfLimbs (mom has limbs)

Why the Bias Term?

Purpose: The bias allows the decision boundary to shift away from the origin.
Without bias: Decision boundary always passes through origin (0, 0, ...)
With bias: Decision boundary can be positioned anywhere
Example:

- Without bias: $2x_1 + 3x_2 = 0$ (line through origin)
- With bias: $2x_1 + 3x_2 - 5 = 0$ (line not through origin)

Analogy:

- Weights determine the *angle* of the decision boundary
- Bias determines its *position*

Mathematical Note: Some implementations treat bias as an extra feature that's always 1:

$$[\text{bias}, x_1, x_2, \ldots, x_n] \to [1, x_1, x_2, \ldots, x_n]$$

Then b becomes just another weight w_0.

2.4 Perceptron in Python

Python Implementation: Perceptron

Basic Perceptron from Scratch:

```python
import numpy as np

def decision(x, w, b):
    """Make prediction: 1 if w*x + b > 0, else 0"""
    return 1 if np.dot(w, x) + b > 0 else 0

def train_perceptron(X, y, learning_rate=1, max_epochs=100):
    """
    Train perceptron on dataset X with labels y

    X: (m, n) array where m = examples, n = features
    y: (m,) array of binary labels (0 or 1)
    """
    m, n = X.shape

    # Initialize weights and bias
    w = np.zeros(n)
    b = 0

    # Training loop
    for epoch in range(max_epochs):
        errors = 0

        # Iterate through each training example
        for i in range(m):
            # Make prediction
            pred = decision(X[i], w, b)

            # Check if prediction is correct
            if pred == y[i]:
                continue  # Correct! No update needed

            # False Negative: y=1 but pred=0
            elif y[i] == 1 and pred == 0:
```

```python
                b += learning_rate
                w += learning_rate * X[i]
                errors += 1

            # False Positive: y=0 but pred=1
            elif y[i] == 0 and pred == 1:
                b -= learning_rate
                w -= learning_rate * X[i]
                errors += 1

        # Check convergence
        if errors == 0:
            print(f"Converged in {epoch + 1} epochs!")
            break

    return w, b

# Example usage
from sklearn.datasets import make_blobs

# Generate linearly separable data
X, y = make_blobs(n_samples=100, n_features=2,
                  centers=2, cluster_std=1.0)

# Train perceptron
w, b = train_perceptron(X, y)

# Make prediction on new point
new_point = np.array([1.0, 2.0])
prediction = decision(new_point, w, b)
print(f"Prediction: {prediction}")
```

Using scikit-learn:

```python
from sklearn.linear_model import Perceptron

# Create and train perceptron
clf = Perceptron(max_iter=100)
clf.fit(X, y)

# Make predictions
predictions = clf.predict(X)
```

3 Feed-Forward Neural Networks (FFNNs)

3.1 Architecture and Terminology

Feed-Forward Neural Network (FFNN)

Also known as: Multi-Layer Perceptron (MLP)
Definition: A neural network where information flows in one direction: from input → through hidden layers → to output (no cycles/loops).
Components:

1. **Input Layer**:
   - Receives the input features $\vec{x} = [x_1, x_2, \ldots, x_n]$
   - Size = number of features
   - Not counted as a "layer" in network depth
2. **Hidden Layers**:
   - Intermediate layers between input and output
   - Extract progressively complex features
   - Can have 0, 1, 2, or many hidden layers
   - Each layer has: weights matrix $\mathbb{W}^{(l)}$ and activation $\vec{\alpha}^{(l)}$
3. **Output Layer**:
   - Produces final prediction $\hat{y}$
   - For binary classification: 1 neuron (probability)
   - For multi-class: k neurons (one per class)

Key Terms:

- **Neurons/Units**: Individual computational nodes in a layer
- **Weights**: Parameters connecting neurons between layers
- **Activations**: Output values of neurons after applying activation function
- **Depth**: Number of hidden layers + output layer
- **Width**: Number of neurons in a layer

Why Multiple Layers?

Single Layer Limitation: Can only learn linear decision boundaries
Multiple Layers Power:

- First hidden layer: Learns simple patterns
- Second hidden layer: Combines simple patterns into complex patterns
- Third hidden layer: Even more abstract patterns
- Output layer: Makes final decision based on high-level patterns

Universal Approximation Theorem:

- A neural network with just ONE hidden layer (with enough neurons) can approximate any continuous function
- In practice: Deeper networks (more layers) learn more efficiently
- Modern deep learning: 10s to 1000s of layers!

Analogy: Building a house

- Layer 1: Recognizes bricks, planks, nails
- Layer 2: Recognizes walls, floors, roof
- Layer 3: Recognizes rooms, structure
- Output: "This is a house" vs "This is not a house"

3.2 Mathematical Formulation

FFNN Forward Pass (Inference)

Notation:

- $\vec{x}$: Input vector (size n_0)
- $\mathbb{W}^{(l)}$: Weight matrix from layer $l-1$ to layer l
- $\vec{b}^{(l)}$: Bias vector for layer l
- $\vec{z}^{(l)}$: Pre-activation values for layer l
- $\vec{\alpha}^{(l)}$: Activation values (post-activation) for layer l
- σ: Activation function
- L: Total number of layers (including output)

Forward Pass Algorithm:
Step 0: Set input

$$\vec{\alpha}^{(0)} = \vec{x}$$

For each layer $l = 1, 2, \ldots, L$:
Step 1: Compute weighted sum (linear transformation)

$$\vec{z}^{(l)} = \mathbb{W}^{(l)}\vec{\alpha}^{(l-1)} + \vec{b}^{(l)}$$

Step 2: Apply activation function (non-linearity)

$$\vec{\alpha}^{(l)} = \sigma(\vec{z}^{(l)})$$

Final output:

$$\hat{y} = \vec{\alpha}^{(L)}$$

Plain English:

1. Start with input features
2. For each layer:
   - Multiply inputs by weights and add bias
   - Apply activation function to get layer output
3. Final layer output is the prediction

Simple 2-Layer Network Example

Network Architecture:

- Input: 4 features $\rightarrow \vec{x} \in \mathbb{R}^4$
- Hidden layer: 3 neurons $\rightarrow \vec{\alpha}^{(1)} \in \mathbb{R}^3$
- Output layer: 2 neurons $\rightarrow \vec{\alpha}^{(2)} \in \mathbb{R}^2$ (for 2-class problem)

Weight Matrices:

- $\mathbb{W}^{(1)}$: Shape (3×4) - connects 4 inputs to 3 hidden neurons
- $\mathbb{W}^{(2)}$: Shape (2×3) - connects 3 hidden neurons to 2 outputs

Forward Pass:
Input: $\vec{x} = [1, 2, 3, 4]^T$
Layer 1:

$$\vec{z}^{(1)} = \mathbb{W}^{(1)}\vec{x} + \vec{b}^{(1)}$$
$$\vec{\alpha}^{(1)} = \text{ReLU}(\vec{z}^{(1)})$$

Layer 2:

$$\vec{z}^{(2)} = \mathbb{W}^{(2)}\vec{\alpha}^{(1)} + \vec{b}^{(2)}$$
$$\vec{\alpha}^{(2)} = \text{softmax}(\vec{z}^{(2)})$$

Output: $\hat{y} = \vec{\alpha}^{(2)} = [0.3, 0.7]^T$
Interpretation: 30% probability class 0, 70% probability class 1 $\rightarrow$ Predict class 1

Understanding Weight Matrix Dimensions

Key Rule: If layer $l-1$ has n_{l-1} neurons and layer l has n_l neurons:

$$\mathbb{W}^{(l)} \text{ has shape } (n_l \times n_{l-1})$$

Why? Matrix multiplication:

$$\underbrace{\mathbb{W}^{(l)}}_{n_l \times n_{l-1}} \times \underbrace{\vec{\alpha}^{(l-1)}}_{n_{l-1} \times 1} = \underbrace{\vec{z}^{(l)}}_{n_l \times 1}$$

Example:

- Input: 784 features (e.g., 28×28 image flattened)
- Hidden: 128 neurons
- $\mathbb{W}^{(1)}$: Shape (128×784)
- Total parameters in $\mathbb{W}^{(1)}$: $128 \times 784 = 100,352$ weights!

Counting Parameters:

- Weights: $n_l \times n_{l-1}$
- Biases: n_l (one per neuron in layer l)
- Total per layer: $n_l \times (n_{l-1} + 1)$

4 Activation Functions

4.1 Why Activation Functions?

The Need for Non-Linearity

Problem: Without activation functions, neural networks collapse to linear models!
Mathematical Proof:

$$\begin{aligned}
\vec{\alpha}^{(1)} &= \mathbb{W}^{(1)}\vec{x} + \vec{b}^{(1)} \\
\vec{\alpha}^{(2)} &= \mathbb{W}^{(2)}\vec{\alpha}^{(1)} + \vec{b}^{(2)} \\
&= \mathbb{W}^{(2)}(\mathbb{W}^{(1)}\vec{x} + \vec{b}^{(1)}) + \vec{b}^{(2)} \\
&= (\mathbb{W}^{(2)}\mathbb{W}^{(1)})\vec{x} + (\mathbb{W}^{(2)}\vec{b}^{(1)} + \vec{b}^{(2)}) \\
&= \mathbb{W}_{\text{combined}}\vec{x} + \vec{b}_{\text{combined}}
\end{aligned}$$

This is just a linear model! Multiple layers provide no benefit.
Solution: Add non-linear activation functions between layers!

$$\vec{\alpha}^{(l)} = \sigma(\mathbb{W}^{(l)}\vec{\alpha}^{(l-1)} + \vec{b}^{(l)})$$

Now the network can learn non-linear patterns!

4.2 Common Activation Functions

Sigmoid Activation

Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- **Range**: $(0, 1)$ - always between 0 and 1
- **Shape**: S-curve (smooth transition)
- **Output interpretation**: Probability (for binary classification)

When to use:

- Output layer for binary classification
- When you need probabilities

Drawbacks:

- **Vanishing gradient**: For very large or small z, gradient ≈ 0
- Slows down learning in deep networks
- Not zero-centered (outputs always positive)

ReLU (Rectified Linear Unit)

Formula:

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Properties:

- **Range**: $[0, \infty)$
- **Shape**: Linear for positive, zero for negative
- **Gradient**: 1 for $z > 0$, 0 for $z < 0$

Advantages:

- **Fast**: Simple to compute
- **No vanishing gradient** (for positive values)
- **Sparse activation**: Many neurons output 0 (efficient)
- **Most popular** for hidden layers in practice!

Drawbacks:

- **Dying ReLU**: Neurons can "die" (always output 0)
- Happens if weights become such that z is always negative

When to use:

- Default choice for hidden layers
- Works well in most cases

Leaky ReLU

Formula:

$$\text{Leaky ReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}$$

where α is a small constant (e.g., 0.01)
Advantage over ReLU:

- Fixes "dying ReLU" problem
- Small gradient (α) for negative values keeps neuron alive

When to use:

- When ReLU neurons are dying
- Alternative to ReLU in deep networks

Softmax Activation

Formula: For output vector $\vec{z} = [z_1, z_2, \ldots, z_k]$:

$$\text{softmax}(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

Properties:

- **Range**: Each output in $(0, 1)$
- **Sum**: All outputs sum to 1: $\sum_i \text{softmax}(\vec{z})_i = 1$
- **Interpretation**: Probability distribution over classes

When to use:

- **Always** use for output layer in multi-class classification
- Converts raw scores (logits) to probabilities

Example: For 3 classes with logits $[2.0, 1.0, 0.1]$:

$$\begin{aligned}\text{softmax}([2.0, 1.0, 0.1]) &= \frac{[e^{2.0}, e^{1.0}, e^{0.1}]}{e^{2.0} + e^{1.0} + e^{0.1}} \\ &\approx [0.66, 0.24, 0.10]\end{aligned}$$

Interpretation: 66% probability class 0, 24% class 1, 10% class 2

Choosing Activation Functions: Quick Guide

Hidden Layers:

- **Default choice**: ReLU
- **If ReLU struggles**: Try Leaky ReLU or other variants (ELU, GELU)
- **Don't use sigmoid**: Too slow, vanishing gradient issues

Output Layer:

- **Binary classification**: Sigmoid (1 neuron)
- **Multi-class classification**: Softmax (k neurons)
- **Regression**: No activation (linear output)

Modern Variants:

- **GELU**: Used in transformers (GPT, BERT)
- **Swish/SiLU**: $f(x) = x \cdot \sigma(x)$
- **Mish**: $f(x) = x \cdot \tanh(\text{softplus}(x))$

5 Training Neural Networks

5.1 Loss Functions

Loss Function for Classification

Purpose: Measure how wrong our predictions are
Cross-Entropy Loss (most common for classification):
Binary Classification:

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Multi-Class Classification:

$$L(\hat{\vec{y}}, \vec{y}) = -\sum_{i=1}^{k} y_i \log(\hat{y}_i)$$

where $\vec{y}$ is one-hot encoded true label, $\hat{\vec{y}}$ is predicted probabilities
Average Loss over Dataset:

$$J(\mathbb{W}, \vec{b}) = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)})$$

Plain English:

- Penalizes confident wrong predictions heavily
- Small penalty for correct predictions
- Goal: Minimize J by adjusting weights and biases

5.2 Gradient Descent and Backpropagation

Training Process Overview

Goal: Find weights $\mathbb{W}$ and biases $\vec{b}$ that minimize loss J
Algorithm: Gradient Descent

1. Start with random weights and biases
2. Repeat until convergence:
   (a) **Forward pass**: Compute predictions for all training examples
   (b) **Compute loss**: Measure how wrong predictions are
   (c) **Backward pass**: Compute gradients (how to change weights)
   (d) **Update weights**: Move weights in direction that reduces loss

Update Rule:

$$\mathbb{W}^{(l)} = \mathbb{W}^{(l)} - \alpha \frac{\partial J}{\partial \mathbb{W}^{(l)}}$$

where α is the learning rate
Plain English:

- Gradient tells us which direction increases loss
- We move in opposite direction (negative gradient)
- Learning rate controls step size

Backpropagation

Definition: Efficient algorithm to compute gradients in neural networks
Key Idea: Use chain rule to compute gradients layer by layer, from output to input
Why it's called "back"propagation:

- Forward pass: Input $\rightarrow$ Output
- Backward pass: Output $\rightarrow$ Input (propagating errors backward)

High-Level Algorithm:

1. Compute output layer error: $\delta^{(L)} = \hat{y} - y$
2. For each layer $l = L-1, L-2, \ldots, 1$:
   - Propagate error: $\delta^{(l)} = (\mathbb{W}^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(\vec{z}^{(l)})$
   - Compute gradient: $\frac{\partial J}{\partial \mathbb{W}^{(l)}} = \delta^{(l)} (\vec{\alpha}^{(l-1)})^T$

Good News: You rarely need to implement this yourself! Modern frameworks (TensorFlow, PyTorch) do it automatically.

5.3 Optimization Algorithms

Stochastic Gradient Descent (SGD)

Standard Gradient Descent: Compute gradient on entire dataset before updating

$$\mathbb{W} = \mathbb{W} - \alpha \frac{1}{m} \sum_{i=1}^{m} \nabla_{\mathbb{W}} L^{(i)}$$

Problem: Slow for large datasets (must process all m examples)
Stochastic Gradient Descent: Update after each example

$$\mathbb{W} = \mathbb{W} - \alpha \nabla_{\mathbb{W}} L^{(i)}$$

Mini-Batch SGD (most common): Update after small batch

$$\mathbb{W} = \mathbb{W} - \alpha \frac{1}{b} \sum_{i \in \text{batch}} \nabla_{\mathbb{W}} L^{(i)}$$

where b is batch size (e.g., 32, 64, 128, 256)
Advantages:

- Much faster than full batch
- Noise in updates helps escape local minima
- Can process datasets larger than memory

Adam Optimizer

Full name: Adaptive Moment Estimation
Key Idea: Adaptive learning rates for each parameter
Algorithm:

1. Compute gradient: $g_t = \nabla_{\mathbb{W}} J$
2. Update first moment (mean): $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
3. Update second moment (variance): $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
4. Bias correction: $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$, $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$
5. Update weights: $\mathbb{W} = \mathbb{W} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$

Advantages:

- Adapts learning rate for each parameter
- Works well with default hyperparameters ($\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$)
- **Most popular optimizer** for deep learning

When to use:

- **Default choice** for most problems
- Especially good for noisy gradients, sparse data

Hyperparameter Tuning

Key Hyperparameters:
Learning Rate (α):

- Too large: Training diverges, loss explodes
- Too small: Training very slow, might get stuck
- Typical values: 0.001, 0.01, 0.0001
- **Most important hyperparameter!**

Batch Size:

- Smaller (32, 64): Noisy updates, better generalization
- Larger (256, 512): Stable updates, faster training (with GPUs)
- Trade-off: Memory vs training dynamics

Number of Epochs:

- Too few: Underfitting
- Too many: Overfitting
- Use validation set to decide when to stop (early stopping)

Network Architecture:

- Number of layers: Start small (1-2 hidden), increase if needed
- Layer size: Often 64, 128, 256, 512 neurons
- More parameters $\rightarrow$ more capacity but harder to train

6 Implementation in Python

6.1 Using TensorFlow/Keras

6.2 Complete FFNN Example: MNIST Digits - Part 1

Problem: Classify handwritten digits (0-9)

Data: 28×28 grayscale images → 784 input features

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# 1. Load and preprocess data
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

# Flatten images: 28x28 -> 784
X_train = X_train.reshape(-1, 784).astype('float32') / 255.0
X_test = X_test.reshape(-1, 784).astype('float32') / 255.0

# 2. Define model architecture
model = keras.Sequential([
    # Input layer (specified implicitly)
    layers.Input(shape=(784,)),

    # Hidden layer 1: 128 neurons with ReLU
    layers.Dense(128, activation='relu'),

    # Optional: Dropout for regularization
    layers.Dropout(0.2),

    # Hidden layer 2: 64 neurons with ReLU
    layers.Dense(64, activation='relu'),

    # Output layer: 10 neurons (one per digit) with softmax
    layers.Dense(10, activation='softmax')
])

# 3. Print model summary
model.summary()

# 4. Compile model (specify optimizer, loss, metrics)
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',  # For integer labels
    metrics=['accuracy']
)
```

6.3 Complete FFNN Example: MNIST Digits - Part 2

Training and Evaluation:

```python
# 5. Train model
history = model.fit(
    X_train, y_train,
    batch_size=128,
    epochs=10,
    validation_split=0.1,  # Use 10% of training for validation
    verbose=1
)

# 6. Evaluate on test set
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")

# 7. Make predictions
```

```python
predictions = model.predict(X_test[:5])
print("Predicted probabilities for first 5 test images:")
print(predictions)

# Get predicted classes
predicted_classes = np.argmax(predictions, axis=1)
print(f"Predicted classes: {predicted_classes}")
print(f"True classes: {y_test[:5]}")
```

Output:

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 128)               100480
dropout (Dropout)            (None, 128)               0
dense_1 (Dense)              (None, 64)                8256
dense_2 (Dense)              (None, 10)                650
=================================================================
Total params: 109,386
Trainable params: 109,386
Non-trainable params: 0
_________________________________________________________________

Epoch 1/10
422/422 [==============================] - 2s 4ms/step
  loss: 0.2876 - accuracy: 0.9161 - val_loss: 0.1423 - val_accuracy: 0.9583
...
Epoch 10/10
422/422 [==============================] - 2s 4ms/step
  loss: 0.0512 - accuracy: 0.9841 - val_loss: 0.0923 - val_accuracy: 0.9742

Test accuracy: 0.9738
```

6.4 Detailed Layer Explanation

Dense Layer:

```python
layers.Dense(units=128, activation='relu')
```

What it does:

- Creates 128 neurons
- Each neuron: $\text{ReLU}(\vec{w} \cdot \vec{x} + b)$
- `units`: Number of neurons
- `activation`: Activation function to use

Dropout Layer:

```python
layers.Dropout(0.2)
```

What it does:

- Randomly sets 20% of activations to 0 during training
- Prevents overfitting
- Only active during training, not evaluation

Common Activations:

- `'relu'`: ReLU activation
- `'sigmoid'`: Sigmoid activation
- `'softmax'`: Softmax (output layer)
- `'tanh'`: Hyperbolic tangent
- `None`: Linear (no activation)

6.5 Data Preprocessing

Key Preprocessing Steps:

1. Normalization (scale features to [0, 1]):

```
X = X.astype('float32') / 255.0  # For image data (0-255)
# Or
X = (X - X.mean()) / X.std()  # Standardization
```

2. Reshaping:

```
# Flatten 2D images to 1D for dense layers
X = X.reshape(-1, 784)  # -1 means "infer this dimension"

# Or expand dimensions for Conv layers
X = X.reshape(-1, 28, 28, 1)  # (samples, height, width, channels)
```

3. One-Hot Encoding (for labels):

```
# If using categorical_crossentropy
from tensorflow.keras.utils import to_categorical
y_train = to_categorical(y_train, num_classes=10)
# [3] -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
```

4. Train/Val/Test Split:

```
from sklearn.model_selection import train_test_split

# Split training into train + validation
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.1, random_state=42
)
```

6.6 Model Training Tips

Monitoring Training:

```
# Plot training history
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='train accuracy')
plt.plot(history.history['val_accuracy'], label='val accuracy')
```

```
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Early Stopping:

```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,  # Stop if no improvement for 5 epochs
    restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    callbacks=[early_stop]
)
```

Model Checkpointing:

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint(
    'best_model.h5',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max'
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    callbacks=[checkpoint]
)
```

7 Evaluation Metrics

7.1 Beyond Accuracy

Why Accuracy Isn't Always Enough

Problem: Accuracy can be misleading for imbalanced datasets
Example: Email spam detection

- 99% emails are not spam, 1% are spam
- Classifier that predicts "not spam" for everything: 99% accuracy!
- But it catches 0% of spam → Useless!

Solution: Use precision, recall, F1-score

Confusion Matrix

For Binary Classification:

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Definitions:

- **TP (True Positive)**: Correctly predicted positive
- **TN (True Negative)**: Correctly predicted negative
- **FP (False Positive)**: Incorrectly predicted positive (Type I error)
- **FN (False Negative)**: Incorrectly predicted negative (Type II error)

Precision / Recall / F1-Score

Precision: Of all positive predictions, how many were correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Plain English: "When the model says positive, how often is it right?"
Recall (Sensitivity): Of all actual positives, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Plain English: "Of all real positive cases, how many did we find?"
F1-Score: Harmonic mean of precision and recall

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Plain English: "Single score that balances precision and recall"
When to care about what:

- **High precision needed**: Spam detection (don't want false alarms)
- **High recall needed**: Cancer detection (can't miss any cases)
- **Balance both**: Use F1-score

7.2 Computing Metrics in Python

```python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

# Get predictions
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

# For binary classification
accuracy = accuracy_score(y_test, y_pred_classes)
precision = precision_score(y_test, y_pred_classes)
recall = recall_score(y_test, y_pred_classes)
f1 = f1_score(y_test, y_pred_classes)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_classes)
print("Confusion Matrix:")
print(cm)

# For multi-class: use average parameter
precision_macro = precision_score(y_test, y_pred_classes,
                                  average='macro')
recall_macro = recall_score(y_test, y_pred_classes,
                            average='macro')
f1_macro = f1_score(y_test, y_pred_classes, average='macro')

```

```python
# Detailed report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_classes))
```

Output:

```
Accuracy: 0.9738
Precision: 0.9745
Recall: 0.9731
F1-Score: 0.9738

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.98       980
           1       0.99      0.99      0.99      1135
           2       0.97      0.97      0.97      1032
           3       0.97      0.97      0.97      1010
           4       0.98      0.98      0.98       982
           5       0.97      0.97      0.97       892
           6       0.98      0.98      0.98       958
           7       0.97      0.96      0.97      1028
           8       0.97      0.97      0.97       974
           9       0.96      0.97      0.97      1009

    accuracy                           0.97     10000
   macro avg       0.97      0.97      0.97     10000
weighted avg       0.97      0.97      0.97     10000
```

8 Common Issues and Solutions

8.1 Overfitting

Overfitting in Neural Networks

Symptoms:

- High training accuracy (¿95
- Much lower test accuracy (¡80
- Large gap between training and validation loss
- Model memorizes training data instead of learning patterns

Causes:

- Too many parameters (large network)
- Too few training examples
- Training too long

Preventing Overfitting

1. **More Data**:
   - Collect more training examples
   - Use data augmentation (rotate, flip, crop images)
2. **Regularization**:
   - **L2 regularization**: Add penalty for large weights
   - **Dropout**: Randomly drop neurons during training
3. **Early Stopping**:
   - Stop training when validation loss stops improving
   - Monitor validation metrics, not training metrics
4. **Reduce Model Complexity**:
   - Fewer layers
   - Fewer neurons per layer
   - Start simple, add complexity only if needed
5. **Cross-Validation**:
   - Use k-fold cross-validation
   - Better estimate of model performance

Example code for regularization:

```
# L2 regularization
layers.Dense(128, activation='relu',
             kernel_regularizer=keras.regularizers.l2(0.01))

# Dropout
layers.Dropout(0.2)  # Drop 20% of neurons
```

8.2 Underfitting

Underfitting

Symptoms:

- Low training accuracy
- Low test accuracy
- Training and validation losses both high

Causes:

- Model too simple (not enough capacity)
- Not enough training
- Poor feature representation

Solutions:

- Increase model complexity (more layers, more neurons)
- Train longer
- Better feature engineering
- Use more complex activation functions
- Reduce regularization

9 What Have You Learned?

Neural Networks: Key Takeaways

1. From Single Neurons to Networks

Perceptron:

- Simplest neural network (one neuron)
- Decision function: $\text{sign}(\vec{w} \cdot \vec{x} + b)$
- Learning: Error-driven updates (correct mistakes)
- Limitation: Only learns linearly separable patterns
- Update rules:
  - False Negative: $\vec{w} = \vec{w} + \vec{x}$, $b = b + 1$
  - False Positive: $\vec{w} = \vec{w} - \vec{x}$, $b = b - 1$

Feed-Forward Neural Networks:

- Multiple layers of neurons
- Can learn non-linear patterns (XOR, circles, etc.)
- Universal function approximators (theoretically)
- Each layer: $\vec{\alpha}^{(l)} = \sigma(\mathbb{W}^{(l)}\vec{\alpha}^{(l-1)} + \vec{b}^{(l)})$

2. Architecture Components

Layers:

- **Input layer**: Receives features (not counted in depth)
- **Hidden layers**: Extract progressively complex features
- **Output layer**: Makes final prediction

Key Parameters:

- **Weights** $\mathbb{W}^{(l)}$: Connect neurons between layers
- **Biases** $\vec{b}^{(l)}$: Shift activation (one per neuron)
- **Activations** $\vec{\alpha}^{(l)}$: Outputs of neurons

3. Activation Functions

Why needed: Without them, networks collapse to linear models!
Common choices:

- **ReLU**: $\max(0, z)$ - Default for hidden layers
- **Sigmoid**: $\frac{1}{1+e^{-z}}$ - Output for binary classification
- **Softmax**: $\frac{e^{z_i}}{\sum_j e^{z_j}}$ - Output for multi-class
- **Leaky ReLU**: Fixes dying ReLU problem

4. Training Process

Forward Pass: Input $\rightarrow$ Hidden layers $\rightarrow$ Output

- Compute activations layer by layer
- Final layer produces prediction

Loss Computation:

- Binary: $L = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$
- Multi-class: $L = -\sum_i y_i \log(\hat{y}_i)$

Backward Pass (Backpropagation):

- Compute gradients using chain rule
- Propagate errors from output to input
- Tells us how to update weights

Weight Update:

- $\mathbb{W} = \mathbb{W} - \alpha \nabla_{\mathbb{W}} J$
- Move weights to reduce loss

5. Optimization

Stochastic Gradient Descent (SGD):

- Update weights on mini-batches (not full dataset)
- Faster, helps escape local minima
- Typical batch sizes: 32, 64, 128, 256

Adam Optimizer:

- Adaptive learning rates per parameter
- Default choice for most problems
- Works well with default settings

Key Hyperparameters:

- Learning rate: 0.001 (typical for Adam)
- Batch size: 64-256
- Number of epochs: Use early stopping

6. Implementation (Keras/TensorFlow)

Basic Template:

```
# 1. Define architecture
model = keras.Sequential([
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

# 2. Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 3. Train
history = model.fit(X_train, y_train,
                    epochs=10, batch_size=128,
                    validation_split=0.1)

# 4. Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test)

# 5. Predict
predictions = model.predict(X_new)
```

7. Evaluation Metrics

Beyond Accuracy:

- **Precision**: $\frac{TP}{TP+FP}$ - When model says yes, how often correct?
- **Recall**: $\frac{TP}{TP+FN}$ - Of all real positives, how many found?
- **F1-Score**: $\frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec}+\text{Rec}}$ - Balance both

When to use:

- Balanced data: Accuracy is fine
- Imbalanced data: Use precision, recall, F1
- Cost-sensitive: Weight false positives vs false negatives differently

8. Common Pitfalls

Overfitting:

- High train accuracy, low test accuracy
- Solutions: More data, dropout, L2 regularization, early stopping

Underfitting:

- Low train and test accuracy
- Solutions: Bigger network, train longer, better features

Vanishing/Exploding Gradients:

- Use ReLU instead of sigmoid
- Proper weight initialization
- Batch normalization

9. Best Practices

1. **Start simple**: 1-2 hidden layers, modest size
2. **Normalize inputs**: Scale to [0,1] or standardize
3. **Use ReLU**: For hidden layers (not sigmoid!)
4. **Use Adam**: Default optimizer choice
5. **Monitor validation**: Catch overfitting early
6. **Use dropout**: 0.2-0.5 for regularization
7. **Batch normalization**: For very deep networks
8. **Data augmentation**: For images especially
9. **Early stopping**: Don't overtrain
10. **Cross-validation**: Better performance estimate

10. Comparison Table

Aspect	Perceptron	FFNN
Layers	1 (input $\rightarrow$ output)	Multiple
Neurons	1	Many
Decision boundary	Linear only	Non-linear
Can solve XOR?	No	Yes
Learning	Error-driven	Gradient descent
Activation	Sign/threshold	ReLU, sigmoid, etc.
Applications	Simple linear problems	Complex real-world tasks

11. Key Formulas to Remember

Perceptron:

$$\text{decision}(x) = \begin{cases} 1 & \text{if } \vec{w} \cdot \vec{x} + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

FFNN Forward Pass:

$$\vec{\alpha}^{(l)} = \sigma(\mathbb{W}^{(l)}\vec{\alpha}^{(l-1)} + \vec{b}^{(l)})$$

Cross-Entropy Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Gradient Descent:

$$\mathbb{W} = \mathbb{W} - \alpha \nabla_{\mathbb{W}} J$$

ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

Softmax:

$$\text{softmax}(\vec{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

12. What's Next?

Beyond Basic Neural Networks:

- **Convolutional Neural Networks (CNNs)**: For images, spatial data
- **Recurrent Neural Networks (RNNs)**: For sequences, time series
- **Transformers**: Modern architecture for NLP (BERT, GPT)
- **Deep Learning**: Very deep networks (100+ layers)
- **Transfer Learning**: Use pre-trained models
- **Autoencoders**: Unsupervised learning
- **GANs**: Generate new data

The Foundation: Master FFNNs first - they're the building blocks for everything else in deep learning!